Git Workflow – Rohit Raj SIngh

Git is a version-control system used to track changes, work safely with branches, collaborate with a team, and maintain project history and helps us to controls diff versions of the softwares.

Git Workflow

1. Clone
git clone <repo-url> creates a local copy of the remote GitHub repository on the developer's machine.

2. Create a Feature Branch
git switch -c feature/day2-profile creates a separate branch for development without directly changing main. The older equivalent is git checkout -b feature/day2-profile.

3. Check Status
git status shows the current branch and identifies modified, staged, or untracked files.

4. Pull / Fetch Latest Changes
git pull origin <branch-name> downloads and integrates remote changes into the current branch. git fetch origin downloads remote information without automatically merging it. Fetching is useful when changes need to be inspected before integration.

5. Make Changes
The developer creates or modifies the required source files, tests, documentation, and other project files.

6. Add
git add . stages the changes so they can be included in the next commit.

7. Commit
git commit -m "Add intern profile data" saves the staged changes as a snapshot in the local Git history. Commit messages should clearly describe the change, for example feat: add intern profile data.

8. Pull / Fetch Before Push
Before pushing, the developer should check whether the remote branch has received new changes. Use git fetch origin to inspect remote updates or git pull origin <branch-name> when those changes need to be integrated. This helps keep the local branch synchronized and reduces conflicts.

9. Push
git push -u origin feature/day2-profile uploads local commits to the remote GitHub repository.

10. Pull Request
A Pull Request proposes merging feature/day2-profile into main. It provides a place for team members to inspect and discuss the changes.

11. Code Review
Reviewers check the code for correctness, readability, maintainability, security, coding standards, and possible bugs. They may comment or request changes before approval.

12. Merge
After review and approval, the feature branch can be merged into main. A command-line merge can be performed with git switch main, git pull origin main, git merge feature/day2-profile, and git push origin main.

Important Git Concepts

- **Merge Conflict:** Occurs when Git cannot automatically combine competing changes. The developer resolves the conflicting sections manually, then runs `git add .` and commits the resolution. Tools may offer Accept Current Change, Accept Incoming Change, or Accept Both Changes.
- **.gitignore:** Specifies files that should not be tracked, such as Python cache files, virtual environments, and environment files. Example: `__pycache__/`, `*.pyc`, `venv/`, `.venv/`, `.env`.

- **README:** Provides project information such as purpose, structure, setup instructions, usage, testing, and workflow.

Overall Workflow

Clone → Branch → Status → Pull/Fetch → Make Changes → Add → Commit → Pull/Fetch → Push → Pull Request → Code Review → Merge

This workflow supports safe feature-based development, keeps local and remote repositories synchronized, maintains meaningful history, and ensures changes are reviewed before being integrated into main.